

**Universidad Mariano Gálvez de Guatemala
Ingeniería En Sistemas de Información
Sede Chiquimulilla**

**Curso: ASEGURAMIENTO DE LA CALIDAD DE SOFTWARE
Docente: CARMELO ESTUARDO MAYÉN MONTERROSO**

**Tarea 3: Guía OWASP, Investigación Planes y Casos de
prueba**

Estudiantes:

Sauly Estefania Cermeño Morales

Carnet:

179-21-8130



Chiquimulilla Santa Rosa 1 de agosto de 2025



GUÍA PRÁCTICA PARA MITIGAR LAS VULNERABILIDADES DEL OWASP TOP 10 – 2021

Dirigida a usuarios no técnicos

Índice

OWASP Top 10 – 2021.	5
Las 10 vulnerabilidades del OWASP Top 10 – 2021.....	5
A01:2021 – Control de acceso roto	5
A02: Fallas Criptográficas	6
A03:2021 – Inyección	7
A04:2021 – Diseño inseguro	8
A05:2021 – Configuración incorrecta de seguridad	9
A06:2021 – Componentes vulnerables y obsoletos	10
A07:2021 – Fallas de identificación y autenticación	11
A08:2021 – Fallos de integridad de software y datos	13
A09:2021 – Errores de registro y supervisión de seguridad.....	14
A10:2021 – Falsificación de solicitudes del lado del servidor (SSRF)	15
Glosario	17
Referencias bibliográficas.....	20

Introducción

Actualmente muchas personas uso de aplicaciones web todos los días para trabajar, comprar, estudiar o comunicarse pero no todas las aplicaciones están bien protegidas algunas tienen fallas de seguridad que pueden ser aprovechadas por personas malintencionadas por esas razones el grupo de OWASP formado por expertos en seguridad informática, identifico las 10 vulnerabilidades mas comunes en sistemas web. Esta guía explica cada una con ejemplos fáciles y muestra cómo pueden prevenirse, incluso sin ser técnico.

OWASP Top 10 – 2021.

El OWASP Top 10 es una lista de las 10 vulnerabilidades de seguridad más comunes en aplicaciones web, creada por una organización sin fines de lucro llamada OWASP “Open Web Application Security Project” la cual busca ayudar a proteger mejor los sitios web y aplicaciones digitales.

Las 10 vulnerabilidades del OWASP Top 10 – 2021

A01:2021 – Control de acceso roto

¿Qué es?

El control de acceso es el sistema que garantiza que los usuarios solo puedan hacer aquello para lo que tienen permiso. Una falla en este control es como tener un edificio donde un empleado puede entrar en la oficina del director, acceder a archivos confidenciales o incluso usar funciones que no le corresponden.

Esto sucede cuando:

- La aplicación permite hacer cosas sin comprobar si la persona tiene permiso.
- Se pueden cambiar datos en la dirección del navegador para acceder a otra información.
- Se puede ver, editar o eliminar cosas de otras personas solo por adivinar su número de identificación.
- Se confunde un usuario común con un administrador.
- La aplicación no cierra bien la sesión o mantiene el acceso por más tiempo del necesario.

¿Cómo prevenirlo?

- Cada persona debe tener acceso solo a lo necesario.
- Probar regularmente si es posible cambiar direcciones web o datos para acceder a funciones prohibidas.
- Evitar mostrar opciones o botones que no correspondan al usuario.

A02: Fallas Criptográficas

¿Qué es?

La criptografía es como el arte de ponerle candado a la información. Se usa para proteger datos importantes, como contraseñas, números de tarjetas, historiales médicos o información privada. Cuando un sistema no protege bien estos datos, se dice que tiene fallas criptográficas. Imagina que escribes tu contraseña en un papel y lo dejas en la calle. Cualquiera podría verlo y usarlo.

Eso es lo que pasa cuando una aplicación guarda o envía datos sin cuidarlos. Es como dejar la puerta abierta.

Esto puede pasar si:

- Los datos viajan por internet sin protección.
- Se usan códigos de seguridad viejos o débiles.
- Se guardan las contraseñas sin protegerlas.
- Se dejan copias de las llaves en lugares inseguros como el código del programa.
- No se aplican reglas básicas de seguridad para la información.

¿Cómo prevenirlo?

- Usar páginas que empiecen con https:// la “s” significa que está protegida.
- No compartir información privada por mensajes o correos.
- No guardar contraseñas en notas o lugares fáciles de encontrar.
- Cambiar las contraseñas de vez en cuando.
- Activar la verificación en dos pasos si una aplicación lo permite (por ejemplo: pedir un código al celular).
- Borrar la información que ya no necesitas guardar.
- No usar programas o aplicaciones que parezcan viejos o poco confiables.
- Mantener siempre actualizados el celular y las aplicaciones.

A03:2021 – Inyección

¿Qué es?

La inyección ocurre cuando una aplicación recibe información escrita por una persona y no la revisa correctamente. Si esa información tiene instrucciones maliciosas, la aplicación puede ejecutarlas sin saber que está siendo engañada esto puede provocar que personas no autorizadas accedan a datos privados, cambien información o controlen partes del sistema ejemplo “Una tienda en línea permite buscar productos escribiendo un nombre. Una persona malintencionada, en lugar de escribir “zapatos”, escribe un conjunto de palabras especiales que engañan a la aplicación para que le muestre los datos de otros clientes. Esto sucede porque la aplicación no separa lo que se escribe del resto de sus instrucciones internas”

Esto sucede cuando:

- La aplicación no revisa ni limpia bien lo que las personas escriben.
- Se mezclan directamente los datos del usuario con las instrucciones internas.
- Se permiten búsquedas sin límites o sin filtros adecuados.
- Se usan herramientas que aceptan cualquier dato sin comprobarlo.

¿Cómo prevenirlo?

- Toda información que escriba una persona debe ser revisada cuidadosamente antes de usarse.
- Los datos deben manejarse por separado de las instrucciones internas de la aplicación.
- Las aplicaciones deben usar herramientas que no permitan mezclar directamente datos con comandos internos.
- Las búsquedas y funciones especiales deben ser lo más cerradas y controladas posible.
- Siempre se deben hacer pruebas antes de publicar una aplicación, para asegurarse de que no se puede engañar fácilmente.

A04:2021 – Diseño inseguro

¿Qué es?

El diseño inseguro sucede cuando una aplicación no está bien planeada desde el inicio para defenderse de los ataques no se trata de errores al escribir el programa, sino de que faltan reglas claras de seguridad desde el comienzo es como construir una casa sin pensar en cerraduras o alarmas. Aunque las paredes estén bien hechas, cualquiera puede entrar por ejemplo “Imagina una aplicación que permite a los usuarios cambiar su contraseña, pero no pide confirmar si de verdad es el dueño de esa cuenta el sistema funciona bien, pero fue mal diseñado, porque no se pensó en evitar que otros se metan y cambien la contraseña”

Este problema aparece cuando:

- No se definen bien los riesgos desde que se empieza a crear el programa.
- El equipo no se detiene a pensar qué cosas malas podrían pasar ni cómo evitarlas.
- No se planean reglas de seguridad desde el inicio, y luego ya es muy tarde para agregarlas.
- No se consulta con personas expertas en seguridad durante el desarrollo.

¿Cómo prevenirlo?

- Planear la seguridad desde el inicio, no dejarla para después.
- Pensar en las posibles situaciones peligrosas y cómo protegerse antes de que pasen.
- Crear una lista de reglas claras de seguridad que se deben seguir siempre.
- Asegurarse de que cada parte del sistema tenga protección adecuada, desde el sitio web hasta la base de datos.

- Dividir bien los niveles del sistema, como si fueran pisos de un edificio, para que lo que pasa en un lugar no afecte todo.
- Limitar lo que puede hacer cada usuario, para evitar abusos

A05:2021 – Configuración incorrecta de seguridad

¿Qué es?

La configuración incorrecta de seguridad ocurre cuando una aplicación no está protegida correctamente desde el inicio o cuando se olvida de asegurar ciertos aspectos importante es como si alguien construyera una casa y dejara las ventanas abiertas o usara cerraduras rotas. Aunque la casa esté bien hecha, cualquier persona puede entrar fácilmente.

Este problema aparece cuando:

- Se dejan habilitadas funciones innecesarias, como servicios o cuentas que no se usan, lo que abre puertas a posibles atacantes.
- Se usan contraseñas predeterminadas o cuentas sin cambiar.
- Los errores revelan información interna detalles sobre el código de la aplicación o el sistema
- La aplicación no está actualizada o las funciones de seguridad no están activadas correctamente.

¿Cómo prevenirlo?

- Configurar todo de manera segura desde el principio. Esto significa usar contraseñas fuertes, cerrar servicios innecesarios y revisar que cada componente esté bien asegurado.
- Eliminar funciones o servicios que no se necesiten. No deje cosas abiertas sin razón.

- Usar un proceso repetible para asegurarse de que todos los entornos de la aplicación estén bien configurados. Desde desarrollo hasta producción, todo debe seguir las mismas reglas de seguridad.
- Seguir buenas prácticas de seguridad, como enviar encabezados de seguridad

A06:2021 – Componentes vulnerables y obsoletos

¿Qué es?

La vulnerabilidad A06:2021 se refiere a un gran problema de seguridad que puede ocurrir cuando los componentes de la aplicación no se actualizan ni se gestionan adecuadamente estos componentes incluyen desde el sistema operativo, servidores y bases de datos hasta bibliotecas o herramientas de programación que se usan en el día a día. Si no se está pendiente de actualizarlos, pueden tener fallos de seguridad conocidos, lo que facilita que los atacantes los exploten.

Este problema aparece cuando:

- No se sabe qué versiones de software se usan en la aplicación, o no se controla bien qué componentes están instalados.
- El software no se actualiza regularmente o se usa software desactualizado que tiene vulnerabilidades conocidas.
- El proceso de aplicar actualizaciones y parches es lento o no ocurre cuando es necesario, dejando a la aplicación vulnerable durante días, semanas o incluso meses.

¿Cómo prevenirlo?

- Mantener un inventario de todos los componentes que usan, tanto del lado del cliente como del servidor el que gestiona los datos.

- Eliminar los componentes que ya no se necesitan o que están obsoletos, como archivos antiguos, servicios sin usar o documentación que ya no tiene utilidad.
- Revisar constantemente las versiones y dependencias de los componentes que se usan en la aplicación, utilizando herramientas para detectar vulnerabilidades.
- Solo descargar componentes desde fuentes oficiales y seguras, y preferir los componentes firmados para evitar que se incluyan versiones modificadas o maliciosas.
- Aplicar actualizaciones y parches de manera rápida, siempre que sea posible. Si no se puede actualizar un componente, se puede buscar una solución temporal para proteger la aplicación mientras se encuentra una actualización.

A07:2021 – Fallas de identificación y autenticación

¿Qué es?

La vulnerabilidad A07:2021 se refiere a fallos en la forma en que una aplicación verifica la identidad de sus usuarios esto incluye los procesos de inicio de sesión y la gestión de sesiones de los usuarios si no se manejan correctamente, los atacantes pueden aprovechar estos fallos para acceder sin permiso a las cuentas de los usuarios, robar información o realizar acciones no autorizadas los fallos comunes incluyen ataques automatizados, contraseñas débiles o procesos de recuperación de contraseñas inseguros.

¿Cuándo ocurre?

- No se usan métodos de autenticación más seguros como la autenticación multifactor
- Las contraseñas predeterminadas, como "admin" o "password1", no se cambian.

- Los procesos de recuperación de contraseñas son débiles, por ejemplo, solo preguntando por el nombre de la madre o una pregunta fácil de adivinar.
- Las contraseñas se almacenan de forma insegura, es decir, sin protección, o con métodos débiles
- Las sesiones de usuario no se gestionan correctamente, lo que significa que después de iniciar sesión, los identificadores de sesión no se invalidan correctamente.

¿Cómo prevenirlo?

- Implementar autenticación multifactor siempre que sea posible esto hace que incluso si alguien adivina o roba la contraseña, no pueda acceder a la cuenta sin el segundo factor como un código enviado al teléfono.
- Evitar contraseñas predeterminadas nunca usar contraseñas como "admin" o "123456" asegurarse de que los usuarios cambien las contraseñas predeterminadas al configurar sus cuentas.
- Revisar las contraseñas de los usuarios y comprobar si son demasiado comunes o débiles, usando herramientas para comparar las contraseñas con listas conocidas de las más usadas.
- Asegurarse de que las contraseñas cumplan con requisitos de seguridad Longitud mínima, combinación de letras, números y caracteres especiales.
- Proteger los procesos de recuperación de contraseñas evitar preguntas de seguridad fáciles y asegurarse de que las rutas de recuperación estén bien protegidas.
- Limitar los intentos fallidos de inicio de sesión para prevenir ataques automáticos, como los de adivinación de contraseñas también, se pueden aplicar retrasos entre los intentos para evitar que un atacante lo intente muchas veces rápidamente.

A08:2021 – Fallos de integridad de software y datos

¿Qué es?

La vulnerabilidad A08:2021 se refiere a fallos en la protección del código y los datos dentro de una aplicación estos fallos ocurren cuando la aplicación no tiene medidas adecuadas para protegerse contra alteraciones maliciosas un ejemplo común es cuando la aplicación utiliza complementos o bibliotecas de fuentes no confiables, o si no verifica adecuadamente las actualizaciones automáticas si los datos o el código no se protegen correctamente, los atacantes pueden modificar el software y hacerlo vulnerable a ataques.

Este problema puede surgir cuando:

- Se usan bibliotecas o componentes de fuentes no confiables esto significa que la aplicación se basa en herramientas o códigos de otras personas o empresas que no han sido verificados como seguros.
- Las actualizaciones automáticas no se verifican correctamente antes de aplicarse esto abre la puerta para que los atacantes envíen actualizaciones falsas.
- Se permiten datos sin verificar los datos que se envían entre los sistemas de la aplicación y los usuarios, como los que contienen información personal, pueden ser modificados por los atacantes si no se protegen correctamente.

¿Cómo prevenirlo?

- Verificar la integridad del software y los datos utilizando herramientas de seguridad como firmas digitales esto asegura que el código proviene de la fuente correcta y no ha sido alterado.
- Usar bibliotecas y dependencias de fuentes confiables asegurarse de que cualquier complemento o código externo provenga de repositorios oficiales y verificados.

- Utilizar herramientas de seguridad como OWASP Dependency Check o OWASP CycloneDX para verificar que las dependencias no contienen vulnerabilidades conocidas.
- Revisar cuidadosamente los cambios de código y configuraciones. Asegurarse de que el código que se añade a la aplicación sea revisado para evitar que se inserten cambios maliciosos.
- Seguir buenas prácticas en la canalización de software asegurarse de que el proceso de desarrollo y distribución de código tenga segregación de funciones y acceso controlado para proteger la integridad del código.
- No enviar datos no verificados a clientes no confiables los datos que se envían entre sistemas deben ser revisados y protegidos para evitar que sean manipulados por los atacantes.

A09:2021 – Errores de registro y supervisión de seguridad

¿Qué es?

Esta vulnerabilidad aparece cuando una aplicación no lleva un buen control o seguimiento de lo que ocurre dentro de ella eso significa que no guarda registros suficientes sobre actividades importantes como inicios de sesión, errores o transacciones, o bien no vigila esos registros para detectar comportamientos extraños sin esa información, es muy difícil saber si alguien está atacando el sistema o si ya lo ha hecho.

¿Cuándo ocurre?

- No se registra información sobre lo que hacen los usuarios.
- Los errores o advertencias no se almacenan o se guardan de forma confusa.
- Nadie revisa esos registros para detectar comportamientos raros.
- Toda la información se guarda solo en la computadora local, sin respaldo.
- No existen alarmas que avisen cuando ocurre algo sospechoso.

- No hay un plan de qué hacer si ocurre un ataque o falla.

¿Cómo prevenirlo?

- Registrar eventos importantes como quién entra al sistema, cuántas veces falla al escribir su contraseña o cuándo realiza acciones importantes.
- Guardar esa información de forma clara, para que los encargados puedan entenderla fácilmente y tomar decisiones.
- Codificar los datos de forma segura para evitar que los atacantes puedan modificar o usar esos registros para hacer daño.
- Para operaciones importantes como mover dinero o cambiar datos personales, guardar una copia segura que no se pueda alterar, como si fuera un libro de contabilidad donde solo se puede escribir, pero no borrar.
- Configurar sistemas de alerta que avisen cuando algo fuera de lo común está pasando.
- Tener un plan claro de qué hacer si hay un ataque o una emergencia esto puede incluir pasos como cerrar temporalmente el acceso, avisar a los responsables o restaurar copias de seguridad.

A10:2021 – Falsificación de solicitudes del lado del servidor (SSRF)

¿Qué es?

Esta vulnerabilidad ocurre cuando una aplicación permite que el usuario escriba una dirección web “URL” y la aplicación va a buscarla sin verificar si esa dirección es confiable o segura así, un atacante puede engañar al sistema para que envíe solicitudes a lugares que normalmente estarían protegidos como redes privadas, servidores internos o servicios en la nube en otras palabras, el atacante usa la aplicación como intermediario para espiar, robar información o atacar otros sistemas.

¿Por qué es peligroso?

- Acceder a datos internos o privados.
- Manipular servicios internos de la empresa.

- Atacar otros sistemas desde una ubicación confiable.
- Explorar la red interna como un espía dentro de casa.
- La amenaza es aún mayor cuando se usan **servicios en la nube**, donde las arquitecturas son más complejas y hay muchos servicios interconectados.

¿Cómo prevenirlo?

A nivel de red

- Dividir las redes: Separar las partes que pueden acceder a recursos externos de las que son internas o sensibles.
- Usar firewalls con reglas estrictas: Solo permitir el tráfico necesario y bloquear todo lo demás por defecto.
- Registrar todo lo que pase por el firewall, tanto lo que se permite como lo que se bloquea.

A nivel de la aplicación

- Validar cuidadosamente cualquier dirección URL que el usuario envíe. Solo aceptar direcciones permitidas lista positiva.
- Evitar mostrar al usuario la respuesta sin procesar de esas direcciones externas.
- Desactivar redirecciones automáticas.
- Ser estrictos con los formatos de URL y mantener consistencia para evitar trampas como la revinculación DNS.
- No confiar en listas de bloqueo listas negras, porque los atacantes conocen formas de evadirlas.

Glosario

A

ACL (Lista de control de acceso): Reglas que indican quién puede entrar o no a ciertas partes de una red o sistema.

Aplicación: Programa que cumple una función, como mostrar una página o hacer una compra en línea.

Autenticación: Proceso donde una persona demuestra que es quien dice ser, por ejemplo, escribiendo una contraseña.

B

Base de datos: Lugar donde se guarda la información que usa una aplicación, como nombres, correos o contraseñas.

C

Control de acceso: Sistema que decide qué puede hacer cada persona dentro de una aplicación.

D

Datos: Información que una persona introduce o que la aplicación usa.

Dependencias: Piezas externas que una aplicación necesita para funcionar, como si fueran ingredientes de una receta.

DNS (Sistema de nombres de dominio): Sistema que traduce nombres de páginas (como google.com) a una dirección que entienden las computadoras.

E

F

Firewall: Barrera que protege a una red, decidiendo qué entra y qué no.

G

H

HTTP (Protocolo de transferencia de hipertexto): Es el idioma que usan las aplicaciones para hablar por internet.

Historial (registro/log): Lista de eventos o actividades que ocurren en una aplicación

I

Inyección: Truco que alguien usa para colarse en el sistema escribiendo comandos donde no debería.

Integridad: Garantía de que la información no ha sido modificada por personas no autorizadas.

J

K

L

Logs (registros): Información guardada sobre lo que ocurre en la aplicación.

M

N

Network (Red): Conjunto de computadoras conectadas entre sí para compartir información.

O

P

Petición HTTP: Cuando una aplicación le pide algo a otra a través de internet.

Puerto: Punto específico donde entra o sale información en un sistema.

Q

R

Redirección: Cuando una aplicación lleva automáticamente a una persona a otra página.

Repositorio: Lugar donde se guarda el código y las herramientas que necesita una aplicación

S

Servidor: Computadora que guarda y envía información a otras.

Sesión: Tiempo que una persona está conectada a una aplicación.

SSRF (Falsificación de solicitudes del lado del servidor): Cuando alguien engaña a la aplicación para que mande información donde no debe.

Serialización: Forma especial de guardar datos que luego se pueden volver a usar o enviar.

Software: Programa o conjunto de programas que hace que una computadora funcione.

T

Transacción: Acción importante dentro de una aplicación, como hacer un pago o cambiar una contraseña.

U

URL: Dirección que se escribe en el navegador para visitar una página

Usuario: Persona que usa una aplicación o sitio web.

V

Validar datos: Revisar que la información que llega a una aplicación sea segura y correcta.

Vulnerabilidad: Falla o debilidad que puede ser usada por alguien para hacer daño o entrar sin permiso.

W

X

Y

Z

Referencias bibliográficas

OWASP top 10:2021. (n.d.). Owasp.org. Retrieved August 1, 2025, from <https://owasp.org/Top10/>

Introduction. (n.d.). Owasp.org. Retrieved August 1, 2025, from https://owasp.org/Top10/A00_2021_Introduction/

Segunda parte:

Realizar una investigación

¿Qué es un Plan de Pruebas?

Un plan de pruebas es un documento fundamental en el ciclo de vida del desarrollo de software. Es el mapa de ruta que guía a todo el equipo de control de calidad. En este plan se detalla el objetivo de las pruebas, las estrategias a seguir, el cronograma del proyecto y los recursos humanos, técnicos y de tiempo que se necesitan para asegurar que el software funcione correctamente antes de su lanzamiento. Es la base para una gestión de calidad exitosa.

¿Por qué es importante tener un Plan de Pruebas?

Contar con un plan de pruebas no es una simple formalidad, es crucial por varias razones que impactan directamente en el éxito de un proyecto:

Organización y Claridad: Un buen plan organiza todo el proceso de pruebas, especificando qué se va a probar, cómo y quién será responsable de cada tarea. Esto minimiza confusiones y asegura que todos los requisitos del software sean verificados de forma sistemática.

Reducción de Costos y Defectos: Al identificar y corregir errores en las primeras etapas del desarrollo, se reducen significativamente los costos de mantenimiento y las fallas post-lanzamiento. Un plan de pruebas detallado ayuda a detectar problemas de manera temprana cuando son más fáciles y baratos de solucionar.

Mejora de la Calidad del Software: La planificación exhaustiva asegura que el producto final cumpla con los estándares de calidad esperados. Al probar a fondo se evitan sorpresas y se ofrece un software más estable y robusto a los usuarios.

Optimización de Recursos: El plan de pruebas permite asignar los recursos de manera eficiente, evitando cuellos de botella y asegurando que la ejecución de las pruebas sea fluida y efectiva.

Tiempo de Comercialización más Rápido: Con un proceso de pruebas claro y bien definido, el desarrollo avanza con mayor eficiencia, lo que permite al equipo lanzar el software al mercado en el menor tiempo posible sin sacrificar la calidad.

Componentes clave de un Plan de Pruebas

Un plan de pruebas se compone de varios elementos que, en conjunto, ofrecen una visión detallada de los procedimientos a seguir. Estos son algunos de los componentes más importantes:

Identificación y Alcance: Se define el nombre del proyecto, el objetivo del plan y el alcance de las pruebas.

Estrategia de Pruebas: Describe el enfoque general para las pruebas, incluyendo los tipos de pruebas que se van a realizar como las funcionales, de rendimiento o de seguridad.

Ambiente de Pruebas: Se detallan los requisitos del entorno de prueba, como hardware, software, configuraciones y herramientas necesarias.

Criterios de Entrada y Salida: Se establecen las condiciones que deben cumplirse para iniciar las pruebas y para considerarlas finalizadas.

Recursos y Cronograma: Se asignan los roles y responsabilidades del equipo, y se establece un calendario con las fechas de inicio y fin de cada fase de prueba.

Riesgos y Mitigación: Se identifican los posibles riesgos del proyecto y se proponen planes para mitigarlos, asegurando que el plan de pruebas sea lo más robusto posible

¿Qué es un Caso de Prueba?

Un caso de prueba es un conjunto de acciones diseñadas para evaluar si un sistema o una característica de software cumple con los requisitos establecidos y funciona como se espera. El propósito principal es validar la funcionalidad del software y asegurar que se alinea con los estándares y las expectativas del cliente. La creación de casos de prueba es una práctica fundamental que también ayuda a identificar defectos o errores en el sistema antes de su implementación.

Generalmente, los casos de prueba son elaborados por equipos de Control de Calidad (QA) o de pruebas. Sirven como instrucciones detalladas y secuenciales para cada proceso de verificación. Una vez que una función del software está lista, se ejecuta un conjunto o una colección de estos casos, conocido como conjunto de pruebas.

Importancia de los Casos de Prueba

Los casos de prueba son cruciales porque definen de manera precisa las acciones necesarias para validar un sistema. Especifican los pasos a seguir, los datos de entrada requeridos y los resultados esperados. Este enfoque estructurado permite a los desarrolladores y a los evaluadores descubrir y corregir errores que de otro modo podrían pasar desapercibidos.

Los beneficios de una estrategia de casos de prueba efectiva son significativos:

Cobertura de Pruebas: Garantizan una validación exhaustiva del software.

Reducción de Costos: Al detectar problemas a tiempo, se minimizan los costos de mantenimiento y soporte.

Reutilización: Los casos de prueba pueden ser reutilizados en futuras iteraciones del software, ahorrando tiempo y esfuerzo.

Aumento de la Calidad: Contribuyen a la mejora de la calidad del software y, por ende, a una mejor experiencia para el usuario final.

En última instancia, el uso de casos de prueba optimiza la operación del negocio. Al entregar productos más confiables, se incrementa la satisfacción y la retención de los clientes, lo que mejora la reputación de la empresa y su rentabilidad.

Componentes y Estructura de un Caso de Prueba

Para que un caso de prueba sea efectivo, debe estar diseñado para evaluar una funcionalidad específica a la vez. El lenguaje utilizado debe ser claro, conciso y consistente. Los componentes esenciales de un caso de prueba incluyen:

ID del Caso de Prueba: Un identificador único que permite agrupar y rastrear las pruebas.

Título o Nombre de la Prueba: Una descripción clara de la funcionalidad que se va a verificar.

Objetivo: Un resumen de lo que la prueba busca validar.

Referencias: Enlaces a documentos clave, como historias de usuario o requisitos de diseño, para contextualizar la prueba.

Prerrequisitos: Las condiciones o dependencias que deben cumplirse antes de ejecutar la prueba.

Configuración de Prueba: La información sobre el entorno, como la versión del software, el sistema operativo y cualquier otra especificación necesaria.

Pasos de Prueba: Una lista detallada de las acciones secuenciales a realizar.

Resultados Esperados: La respuesta o el comportamiento que el sistema debe mostrar después de ejecutar los pasos.

un plan de pruebas

prueba

Discover AIIntegrateAutomate

Invite / 1

Main table

New taskSearchPersonFilterSortHideGroup by

To-Do

	Task		Owner	Status	Due date	
☐	Task 1			Working on it	Jul 31	
☐	Task 2			Done	Aug 1	
☐	Task 3			Stuck	Aug 2	
☐	+ Add task					

Jul 31 + Aug 2

Completed

	Task		Owner	Status	Due date	
☐	+ Add task					

casos de prueba dentro del trabajo

Informe de errores de casos de prueba del proyecto

Name:	ABC Systems
Project ID:	W 20938
Application ID/Name	ABC CRM Software
From Report Date:	17 June 2020
Report Date:	19 June 2020
Complete By(Milestone):	21 July 2020
Manager:	Adam James
QA Manager:	William Nichols

Summary

Total Test Cases	3
Executed	2
Pass	1
Fail	1
Not Executed	1

Functional Testing

Total Case ID	Description	Pass/Fail/Not Executed	Test Date	Responsible Developer	Responsible Tester	Comment	Additional Comment (Other than QA Team)
01	Valid Login	Pass	18-june-2020	Developer 1	Tester 1	Login Successful	Add Text Here
02	Login Error on Invalid Password	Fail	18-june-2020	Developer 1	Tester 2	Incorrect Error Message on Failure	Add Text Here
03	Add Text Here	Not Executed	18-june-2020	Developer 3	Tester 2	NA	Add Text Here

This slide is 100% editable. Adapt it to your needs and capture your audience's attention.

Plan de Pruebas para Validación de Catálogo

1. Objetivo

Validar que el sistema de catálogo permita realizar las operaciones de creación, edición y eliminación de registros de manera funcional, segura y consistente, cumpliendo con los requisitos establecidos.

2. Alcance

El plan cubre las siguientes funcionalidades del catálogo:

- Creación de nuevos registros.
- Edición de registros existentes.
- Eliminación de registros.
- Validación de la interfaz de usuario y la lógica de negocio asociada.

Se excluyen pruebas de rendimiento, seguridad avanzada y compatibilidad con múltiples navegadores, salvo que se especifique.

3. Supuestos

- El sistema está desplegado en un entorno de pruebas estable.
- Se cuenta con acceso a datos de prueba representativos.
- Los requisitos funcionales del catálogo están documentados y disponibles.

4. Tipos de Pruebas

- **Pruebas Funcionales:** Verificar que las operaciones de crear, editar y eliminar funcionen según los requisitos.
- **Pruebas de Validación de Datos:** Asegurar que los datos ingresados sean validados correctamente.
- **Pruebas de Interfaz de Usuario:** Confirmar que la interfaz sea intuitiva y responda adecuadamente.
- **Pruebas de Integración:** Validar que las operaciones del catálogo interactúen correctamente con la base de datos.
- **Pruebas de Regresión:** Asegurar que las nuevas funcionalidades no afecten las existentes.

5. Casos de Prueba

5.1. Creación de Registros

ID	Descripción	Datos de Entrada	Resultado Esperado	Criterio de Aceptación
CP-CR-01	Crear un registro con datos válidos	Nombre: "Producto A", Precio: 100, Categoría: "Electrónica"	Registro creado y mensaje de confirmación mostrado	Registro aparece en el catálogo
CP-CR-02	Intentar crear un registro con campos obligatorios vacíos	Nombre: "", Precio: 100	Mensaje de error: "El nombre es obligatorio"	Registro no se crea
CP-CR-03	Crear un registro con datos inválidos	Precio: -50	Mensaje de error: "El precio debe ser positivo"	Registro no se crea
CP-CR-04	Crear un registro con longitud de campo excedida	Nombre: (texto de 256 caracteres)	Mensaje de error: "El nombre excede el límite de caracteres"	Registro no se crea

5.2. Edición de Registros

ID	Descripción	Datos de Entrada	Resultado Esperado	Criterio de Aceptación
CP-ED-01	Editar un registro existente con datos válidos	Cambiar Nombre: "Producto A" a "Producto B"	Registro actualizado y mensaje de confirmación mostrado	Cambios reflejados en el catálogo
CP-ED-02	Intentar editar un registro con datos inválidos	Precio: -20	Mensaje de error: "El precio debe ser positivo"	Registro no se actualiza
CP-ED-03	Intentar editar un registro inexistente	ID: 999 (no existe)	Mensaje de error: "Registro no encontrado"	No se realiza ninguna acción
CP-ED-04	Editar un registro sin cambios	Mantener datos originales	Mensaje: "No se realizaron cambios"	Registro permanece igual

5.3. Eliminación de Registros

ID	Descripción	Datos de Entrada	Resultado Esperado	Criterio de Aceptación
CP-EL-01	Eliminar un registro existente	Seleccionar registro con ID: 1	Registro eliminado y mensaje de confirmación mostrado	Registro no aparece en el catálogo
CP-EL-02	Intentar eliminar un registro inexistente	ID: 999 (no existe)	Mensaje de error: "Registro no encontrado"	No se realiza ninguna acción
CP-EL-03	Confirmar eliminación con diálogo de confirmación	Seleccionar registro, confirmar eliminación	Registro eliminado	Registro no aparece en el catálogo
CP-EL-04	Cancelar eliminación en diálogo de confirmación	Seleccionar registro, cancelar eliminación	Registro no se elimina	Registro permanece en el catálogo

5.4. Pruebas de Interfaz de Usuario

ID	Descripción	Acción	Resultado Esperado	Criterio de Aceptación
CP-UI-01	Verificar acceso a formulario de creación	Hacer clic en "Crear Registro"	Formulario de creación visible	Formulario se muestra correctamente
CP-UI-02	Verificar habilitación de botón "Guardar"	Ingresar datos válidos en formulario	Botón "Guardar" habilitado	Botón responde al estado del formulario
CP-UI-03	Verificar mensaje de error en tiempo real	Ingresar precio negativo	Mensaje de error aparece sin enviar formulario	Validación en tiempo real funciona

6. Criterios de Aceptación Generales

- Todas las operaciones crear, editar, eliminar deben completarse sin errores críticos.
- Los mensajes de error deben ser claros y específicos.
- Los datos deben persistir correctamente en la base de datos.
- La interfaz debe ser responsiva y no presentar fallos visuales.
- El sistema debe mantener la consistencia de los datos tras cada operación.

7. Herramientas y Recursos

- **Herramientas de Pruebas:** Postman para APIs, si aplica, Selenium para UI automatizada, JUnit o similar para pruebas unitarias.
- **Entorno:** Servidor de pruebas con base de datos configurada.
- **Datos de Prueba:** Conjunto de registros predefinidos y datos límite (valores nulos, negativos, largos, etc..).

8. Cronograma

- Diseño de casos de prueba: 1 día.
- Ejecución de pruebas funcionales y de UI: 3 días.
- Ejecución de pruebas de integración y regresión: 1 día.
- Reporte de resultados y corrección de defectos: 1 día.

9. Responsables

- **Analista de Pruebas:** Diseña y ejecuta los casos de prueba.
- **Desarrollador:** Corrige defectos identificados.
- **Líder de Proyecto:** Aprueba el plan y supervisa el proceso.

10. Gestión de Defectos

- Los defectos se registrarán en una herramienta como Jira o Excel.
- Prioridad: Crítica bloquea funcionalidad, Alta afecta funcionalidad principal, Media afecta funcionalidad secundaria, Baja cosmético.
- Se re ejecutarán las pruebas afectadas tras la corrección de defectos.

11. Entregables

- Documento de plan de pruebas este documento.
- Reporte de resultados de pruebas con defectos encontrados y estado final.
- Evidencia de ejecución capturas de pantalla, logs.

Referencias bibliográficas

Hashemi-Pour, C., & Brush, K. (2024, December 19). *What is A test case?* Search Software Quality; TechTarget. <https://www.techtarget.com/searchsoftwarequality/definition/test-case>

What is a test plan: Importance, components, how to create test plan. (n.d.).

BrowserStack. Retrieved August 1, 2025, from <https://www.browserstack.com/test-management/features/test-run-management/what-is-test-plan>